

AURA Forge Impact and Tool-Evolution Report

Current evidence packet · 3 tracked Markdown source(s) · 2026-09-01

Control vs AURA Forge Impact

Source: docs/capstone/control-vs-aura-impact.md

This document records measured impact for the three representative capstone scenarios. It uses only local PRE-AURA control evidence and post-AURA adaptive-route evidence already recorded in docs/capstone/control-baseline-comparison.md and docs/capstone/aura-results.md.

No company-wide, production, or future-scenario savings are inferred from these three measurements.

Measurement Scope

Label	Meaning
MEASURED	Directly observed in the three local capstone scenario runs.
PROJECTED	Not used for savings claims in this document.

All values below are MEASURED unless explicitly labeled otherwise.

Scenario Impact

AF-LOW-001

Metric	PRE-AURA Control	AURA	Change
Quality score	14/16 FAIL	16/16 PASS	+2 points
Route appropriateness	2/4	4/4	+2 points
Successful-route cost	\$0.6066006	\$0.3377550	-\$0.2688456 (-44.32%)
Measured model-time cycle proxy	89.398 s	45.969 s	-43.429 s (-48.58%)
Reviewer execution-time proxy	14.910 s	15.888 s	+0.978 s (+6.56%)
Model roles	5	2	-3 (-60.00%)
Human checkpoints	2	0	-2 (-100.00%)

LOW shows the clearest adaptive-autonomy gain. The documentation-only change retained review and deterministic verification while removing unnecessary Planner, Tester model, Project Manager, and human-checkpoint overhead.

AF-MEDIUM-001

Metric	PRE-AURA Control	AURA	Change
Quality score	15/16 PASS	16/16 PASS	+1 point
Successful-route cost	\$0.9093231	\$0.7300815	-\$0.1792416 (-19.71%)
Measured model-time cycle proxy	193.330 s	109.502 s	-83.828 s (-43.36%)
Reviewer execution-time proxy	36.680 s	32.490 s	-4.190 s (-11.42%)
Model roles	5	3	-2 (-40.00%)
Human checkpoints	2	1	-1 (-50.00%)

MEDIUM shows a smaller but still meaningful reduction. AURA preserved implementation, independent review, focused test evidence, and final human approval while removing Planner, plan approval, and Project Manager for a bounded utility/test change.

The provider session-limit failure before one MEDIUM Tester attempt remains classified separately as infrastructure failure and is not counted as successful-route cost.

AF-HIGH-001

Metric	PRE-AURA Control	AURA	Change
Quality score	15/16 PASS	16/16 PASS	+1 point
Successful-route cost	\$1.1753241	\$1.1061042	-\$0.0692199 (-5.89%)
Measured model-time cycle proxy	248.666 s	158.510 s	-90.156 s (-36.26%)
Reviewer execution-time proxy	71.825 s	23.370 s	-48.455 s (-67.46%)
Model roles	5	5	0 (0.00%)
Human checkpoints	2	2	0 (0.00%)

HIGH deliberately preserved full governance because the scenario touched evaluation and MCP/governance-sensitive paths. AURA did not optimize away Planner, human approvals, Tester, or Project Manager for this sensitive scenario.

Aggregate Measured Impact

Metric	PRE-AURA Total	AURA Total	Change
Quality score	44/48	48/48	+4 points
Successful-route cost	\$2.6912478	\$2.1739407	-\$0.5173071 (-19.22%)
Measured model-time cycle proxy	531.394 s	313.981 s	-217.413 s (-40.91%)
Reviewer execution-time proxy	123.415 s	71.748 s	-51.667 s (-41.86%)
Model roles	15	10	-5 (-33.33%)
Human checkpoints	6	3	-3 (-50.00%)

Five-Metric Baseline Coverage

The grader requested quality, review latency, defect rate, cycle time, and cost. The evidence supports the following bounded comparison:

Required metric	PRE-AURA	AURA	Calculation and limitation
Quality	44/48	48/48	Sum of the same four-dimension rubric across three scenarios.
Review latency proxy	123.415 s	71.748 s	Sum of Reviewer execution duration. Change: (71.748 - 123.415) / 123.415 × 100 = -41.86%. It excludes queue and human waiting time.
Defect-rate proxy	1 of 3 scenarios failed the locked rubric, or 33.33%	0 of 3 failed, or 0%	1 / 3 × 100 = 33.33%; AURA is 0 / 3 × 100 = 0%. This is a scenario-level rubric-failure proxy, not production defect density.
Cycle-time proxy	531.394 s	313.981 s	Sum of measured model duration. Change: (313.981 - 531.394) / 531.394 × 100 = -40.91%. Complete human wait time was not consistently available.
Cost	\$2.6912478	\$2.1739407	Change: \$2.1739407 - \$2.6912478 = -\$0.5173071; -\$0.5173071 / \$2.6912478 × 100 = -19.22%.

The proxy labels are deliberate. No unavailable wall-clock or production-defect measurement is presented as direct evidence.

Thesis Assessment

The adaptive-autonomy thesis is supported by the measured capstone evidence.

AURA Forge reduced route weight when deterministic risk classification showed lower risk, while retaining strong governance for HIGH. LOW saw the largest reduction in agentic and human overhead while quality improved. MEDIUM saw a smaller reduction while preserving engineering rigor. HIGH saw little cost reduction because strong governance was intentionally retained.

This evidence is bounded to the three representative scenarios. It does not project savings across all FitGPT work, all LaunchCode work, or future production operations.

AURA Forge Impact and Tool-Evolution Report

Current evidence packet · 3 tracked Markdown source(s) · 2026-09-01

Controlled Tool-Evolution Fault Injection

Source: docs/capstone/tool-evolution-drill.md

Drill ID: controlled-fault-001

This was an intentional capstone drill, not an accidental discovery.

Attempted Change

Commit 7c666e5 intentionally added:

```
continue-on-error: true
```

to the permanent policy-tests job in .github/workflows/ci.yml.

Why It Was Plausible

The workflow already uses continue-on-error: true for the advisory AI review job. A legitimate-seeming but unsafe tool evolution is to copy that non-blocking behavior to another job to reduce friction.

Exact Breakage

The permanent policy gate became non-blocking. That weakens the core governance guarantee that policy and MCP runtime failures must fail CI.

Gate That Caught It

scripts/check-pipeline-integrity.py

Failure evidence:

```
{
  "failures": [
    "policy-tests must not use continue-on-error"
  ],
  "schema_version": "pipeline-integrity-v1",
  "status": "FAIL",
  "workflow": ".github/workflows/ci.yml"
}
```

Local evidence:

- .eval-artifacts/capstone/tool-evolution-drill/controlled-fault-001/pipeline-integrity-failure.json
- .eval-artifacts/capstone/tool-evolution-drill/controlled-fault-001/pipeline-integrity-failure-exit-code.txt

Diagnosis

The integrity checker correctly distinguishes advisory review from permanent gates. Advisory review may be non-blocking; policy-tests must not be.

Fix

Remove continue-on-error: true from policy-tests and rerun the same integrity gate.

Final Result

The repaired workflow passes the integrity checker again. The local history intentionally preserves:

- the fault commit;
- the failing gate evidence;
- the fixing commit.

The broken state was never pushed.

AURA Forge Impact and Tool-Evolution Report

Current evidence packet · 3 tracked Markdown source(s) · 2026-09-01

Timeout, Retry, and Budget Enforcement

Source: docs/capstone/reliability-controls.md

Purpose

The grader reported that timeout, retry, and budget controls were described but not demonstrated. eval/reliability_controls.py now makes each decision deterministic and testable from recorded execution evidence.

The control is separate from the frozen aura-router-v1 measurement. It does not rewrite historical scenario results.

Public Decision Contract

evaluate_execution_limits(limits, evidence) accepts:

- a positive timeout;
- an attempt limit of at least one;
- a positive model-cost limit;
- an explicit retryable-failure allow-list;
- the current attempt number, elapsed seconds, recorded cost, and outcome.

It returns one fail-closed decision:

Decision	Meaning
COMPLETED	The attempt succeeded within every limit.
RETRY_ALLOWED	The failure is allow-listed and another bounded attempt remains.
RETRY_LIMIT_REACHED	The configured attempt count is exhausted.
TIMEOUT_EXCEEDED	Elapsed time reached or exceeded the limit.
BUDGET_EXCEEDED	Recorded cost reached or exceeded the limit.
ESCALATION_REQUIRED	The failure is not explicitly retryable.

Timeout and budget checks take precedence over retry permission. A retryable provider failure cannot continue once either boundary is reached.

Demonstrated Cases

eval/test_reliability_controls.py proves:

- one retryable failure can receive one bounded retry;
- a third attempt is denied when max_attempts is two;
- timeout stops a retryable failure;
- budget exhaustion stops a retryable failure;
- a policy denial escalates without retry.

Run:

```
pytest -q -p no:cacheprovider eval/test_reliability_controls.py
```

The permanent GitHub evaluation gate includes this test.

Limitation

This is deterministic policy enforcement over recorded attempt evidence. It does not claim that a paid model call was rerun for this revision, and it does not alter the locked historical holdouts.